

Milestone 4: Test Plan & Script

Group 10: Felix Botero, Mariah Weldon, Prisca Louis, Erik Gijlstra

Payment Processing Page – Front End – Mariah

Test Script PUI 1: Required Fields

Test Case ID: UI_TC_001

Description: Validate that required fields cannot be submitted empty.

Preconditions: Payment Processing page is loaded and active.

Test Steps:

1. Leave **Card Number**, **Expiry Date**, **CVV**, and **Amount** blank.
2. Click **Pay**.

Expected Result:

Validation messages appear for all empty fields; submit button remains disabled.

Actual Result:

Pass/Fail:

Test Script PUI 2: Card Number Format Validation

Test Case ID: UI_TC_002

Description: Ensure card number field accepts only numeric input and passes Luhn validation.

Preconditions: Payment Processing page is loaded.

Test Steps:

1. Enter invalid card number 4111 1111 1111 1112.
2. Enter valid card number 4111 1111 1111 1111.
3. Click **Pay**.

Expected Result:

Error message appears for invalid number (“Invalid card number”).

Valid number is accepted; no error shown.

Actual Result:

Pass/Fail:

Test Script PUI 3: Expiry Date Validation

Test Case ID: UI_TC_003

Description: Verify that the expiry field blocks past or improperly formatted dates.

Preconditions: Current date is October 2025.

Test Steps:

1. Enter 01/20 (expired).
2. Enter 13/28 (invalid month).
3. Enter 07/28 (valid).
4. Click **Pay**.

Expected Result:

Errors appear for expired and invalid months.

Valid date passes with no error.

Actual Result:

Pass/Fail:

Test Script PUI 4: CVV Validation

Test Case ID: UI_TC_004

Description: Validate correct CVV length for card type.

Preconditions: Payment form auto-detects card type from BIN.

Test Steps:

1. Enter Visa card with 2-digit CVV.
2. Enter Visa card with 3-digit CVV.
3. Enter Amex card with 3-digit CVV.
4. Enter Amex card with 4-digit CVV.

Expected Result:

Visa requires 3 digits; Amex requires 4.

Invalid lengths trigger “Incorrect CVV length” error.

Actual Result:

Pass/Fail:

Test Script PUI 5: Amount Field Validation

Test Case ID: UI_TC_005

Description: Ensure amount input is positive and limited to two decimal places.

Preconditions: Payment page open.

Test Steps:

1. Enter 0.
2. Enter -1.
3. Enter 10.999.
4. Enter 49.95.
5. Click **Pay**.

Expected Result:

Errors for zero, negative, or 3-decimal inputs.

49.95 accepted successfully.

Actual Result:

Pass/Fail:

Test Script PUI 6: Successful Payment Flow

Test Case ID: UI_TC_006

Description: Verify full payment submission and confirmation behavior.

Preconditions: API endpoint /api/payments returns 201 success with { "id": "P-9001", "status": "approved", "amount": 49.95 }.

Test Steps:

1. Enter valid payment data.
2. Click **Pay**.

Expected Result:

Success toast displays “Payment approved • \$49.95 • Ref P-9001”.

Form resets after confirmation.

Actual Result:

Pass/Fail:

Test Script PUI 7: Amount Exceeds Available

Test Case ID: UI_TC_007

Description: Show error when entered amount exceeds available balance.

Preconditions: API returns 422 AMOUNT_EXCEEDS_AVAILABLE for ORD1002 (remaining \$30.00).

Test Steps:

1. Enter **Order ID** ORD1002, **Amount** 40.00.
2. Click **Pay**.

Expected Result:

Error banner: "Amount exceeds available (\$30.00)."

No success toast; form remains enabled.

Actual Result:

Pass/Fail:

Test Script PUI 8: No Authorization Found

Test Case ID: UI_TC_008

Description: Display message when payment attempted without valid authorization.

Preconditions: API returns 409 NO_APPROVED_AUTH for ORD2001.

Test Steps:

1. Enter **Order ID** ORD2001, **Amount** 25.00.
2. Click **Pay**.

Expected Result:

Error banner: "No approved authorization found for this order."

No state change in UI.

Actual Result:

Pass/Fail:

Test Script PUI 9: Server/Network Error Handling

Test Case ID: UI_TC_009

Description: Verify behavior during network or server errors.

Preconditions: API returns 500 error or network offline.

Test Steps:

1. Enter valid inputs.

2. Click **Pay**.

Expected Result:

Toast displays “Server error — please try again later.”

Inputs preserved for retry.

Actual Result:

Pass/Fail:

Test Script PUI 10: Double-Submit Guard

Test Case ID: UI_TC_010

Description: Prevent duplicate payment submissions from rapid clicking.

Preconditions: API processes only one successful payment.

Test Steps:

1. Enter valid data.
2. Double-click **Pay** quickly.

Expected Result:

Only one API call sent; button disabled after first click.

Single success toast appears.

Actual Result:

Pass/Fail:

Test Script PUI 11: Responsive Layout (Bootstrap)

Test Case ID: UI_TC_011

Description: Verify layout and readability across viewport sizes.

Preconditions: App open in browser dev tools.

Test Steps:

1. Resize window to 320px, 768px, and 1024px.

Expected Result:

Inputs and buttons align properly; text remains readable; no overflow or clipping.

Actual Result:

Pass/Fail:

Test Script PUI 12: Accessibility (Keyboard & Screen Reader)

Test Case ID: UI_TC_012

Description: Check that form is usable via keyboard and accessible.

Preconditions: Screen reader active (optional).

Test Steps:

1. Tab through fields and press **Enter** to submit.

Expected Result:

All elements receive focus; ARIA labels announce correctly; form submits with Enter key.

Actual Result:

Pass/Fail:

Warehouse Management Page – Front End – Erik

Test Script WUI 1: Required Fields

Test Case ID: UI_TC_001

Description: Verify validation for empty fields.

Preconditions: Warehouse Settlement page open.

Test Steps:

1. Leave both fields blank and click **Settle**.

Expected Result:

Validation messages for Order ID and Amount; submit blocked.

Actual Result:

Pass/Fail:

Test Script WUI 2: Amount Must Be Positive and 2 Decimals Max

Test Case ID: UI_TC_002

Description: Check client-side validation for invalid amounts.

Preconditions: Warehouse UI open.

Test Steps:

1. Enter Order ID=ORD1001, Amount=0. Submit.
2. Enter Amount=-1. Submit.
3. Enter Amount=10.999. Submit.

Expected Result:

Client blocks submit; error message “Enter a positive amount with at most 2 decimals.”

Actual Result:**Pass/Fail:****Test Script WUI 3: Success Path UX**

Test Case ID: UI_TC_003

Description: Ensure UI shows confirmation on successful settlement.

Preconditions: API returns 200 for partial settlement.

Test Steps:

1. Enter ORD1001, 49.95, click **Settle**.

Expected Result:

Success toast “Settlement recorded. Remaining \$100.00.”

Actual Result:**Pass/Fail:****Test Script WUI 4: Exceeds Available UX**

Test Case ID: UI_TC_004

Description: Show error when amount > remaining authorized.

Preconditions: API returns 422 AMOUNT_EXCEEDS_AVAILABLE.

Test Steps:

1. Enter ORD1002, 40.00, click **Settle**.

Expected Result:

Error banner “Amount exceeds available (\$30.00).”

Actual Result:

Pass/Fail:

Test Script WUI 5: No Authorization UX

Test Case ID: UI_TC_005

Description: Show message for order without authorization.

Preconditions: API returns 409 NO_APPROVED_AUTH.

Test Steps:

1. Enter ORD2001, 25.00, click **Settle**.

Expected Result:

Error banner “No approved authorization found for this order.”

Actual Result:

Pass/Fail:

Test Script WUI 6: Server/Network Error UX

Test Case ID: UI_TC_006

Description: Show graceful error on 500 response.

Preconditions: API temporarily returns 500.

Test Steps:

1. Enter valid values and submit.

Expected Result:

Toast “Something went wrong – try again.” Form values preserved.

Actual Result:

Pass/Fail:

Test Script WUI 7: Integration – Auto-Refresh After Success

Test Case ID: UI_TC_007

Description: Verify orders list refreshes after settlement.

Preconditions: UI connected to API.

Test Steps:

1. Submit successful settlement for ORD1001.
2. Open Orders tab.

Expected Result:

Order status shows “SETTLED” and updated timestamp.

Actual Result:

Pass/Fail:

Order Lookup Page – Front End – Prisca

Test Script 1: Verify Display of Current Orders

- **Test Case ID:** PO_TC_001
- **Description:** Validate that current orders (created within the last 30 days) are displayed on the Current tab.
- **Preconditions:** The Current & Past Orders application is open; the backend API is running with seeded order data.

Test Steps:

1. Open the Current & Past Orders UI.
2. Click the “**Current**” tab.
3. Observe the list of displayed orders.

Expected Result:

Only orders created within the last 30 days should appear under the “Current” tab. Each order should display the order number, date, total amount, status, and payment method.

Actual Result: (To be filled in by the tester)

Pass/Fail: (To be marked by the tester)

Test Script 2: Verify Display of Past Orders

- **Test Case ID:** PO_TC_002
- **Description:** Ensure that past orders (older than 30 days) are displayed correctly when selected.
- **Preconditions:** The frontend application is running and connected to the backend API.

Test Steps:

1. Launch the Current & Past Orders UI.
2. Click on the **“Past”** tab.
3. Observe the list of displayed orders.

Expected Result:

Only orders older than 30 days should be displayed under the “Past” tab with correct order details and payment info.

Actual Result: (To be filled in by the tester)

Pass/Fail: (To be marked by the tester)

Test Script 3: Validate Connection Between Frontend and Backend

- **Test Case ID:** PO_TC_003
- **Description:** Confirm that the frontend retrieves order data from the backend successfully.
- **Preconditions:** Both the frontend and backend servers are running.

Test Steps:

1. Open the application.
2. Observe the Orders page as it loads.
3. Verify that order data appears automatically without manual input.

Expected Result:

The frontend displays order information that matches the database entries provided by the backend service.

Actual Result: (To be filled in by the tester)

Pass/Fail: (To be marked by the tester)

Test Script 4: Validate Table Structure and Formatting

- **Test Case ID:** PO_TC_004
- **Description:** Verify that the table layout, columns, and formatting are correctly displayed using Bootstrap styling.
- **Preconditions:** The frontend is open and displays order data.

Test Steps:

1. Observe the table on the Orders page.
2. Verify column headers: “Order #”, “Date”, “Total”, “Status”, and “Payment”.
3. Check that rows are aligned properly and styled using Bootstrap classes.

Expected Result:

All columns are visible, text is properly aligned, and Bootstrap formatting (table hover, borders, nav-pills) is correctly applied.

Actual Result: (To be filled in by the tester)

Pass/Fail: (To be marked by the tester)

Test Script 5: Validate System Behavior When Backend is Offline

- **Test Case ID:** PO_TC_005
- **Description:** Check how the frontend behaves when the backend API is unavailable.
- **Preconditions:** Backend server is not running.

Test Steps:

1. Stop the backend server.
2. Refresh the frontend page.
3. Observe the displayed content or any error message.

Expected Result:

The frontend should show a warning message (e.g., “Backend not reachable, using mock data”) and continue displaying mock orders without crashing.

Actual Result: (To be filled in by the tester)

Pass/Fail: (To be marked by the tester)

Test Script 6: Verify Navigation and Page Load

- **Test Case ID:** PO_TC_006
- **Description:** Ensure that the Current & Past Orders page loads successfully without console errors.
- **Preconditions:** The frontend application is deployed locally and ready for testing.

Test Steps:

1. Launch the application URL.
2. Observe the page load speed and any console errors.

Expected Result:

The page should load smoothly within a few seconds and display the orders table without any JavaScript or network errors.

Actual Result: (To be filled in by the tester)

Pass/Fail: (To be marked by the tester)

Back End Functions – Erik

Test Script BE1: Create Settlement ≤ Authorized (first partial)

Test Case ID: SET_TC_001

Description: Record a partial settlement that is strictly less than the remaining authorized amount.

Preconditions: ORD1001 authorized for \$149.95; no prior settlements.

Test Steps:

1. Send POST /api/settlements with body { "orderId": "ORD1001",
"amount": 49.95 }.

Expected Result:

200 OK with status=PARTIALLY_SETTLED, availableToSettle=100.00, and a new settlement recorded for \$49.95.

Actual Result:

Pass/Fail:

Test Script BE 2: Exact Settle to Zero (fully settled)

Test Case ID: SET_TC_002

Description: Finish settling the remaining authorized amount exactly.

Preconditions: Continuation of A1 – ORD1001 remaining = \$100.00.

Test Steps:

1. POST /api/settlements with { "orderId":"ORD1001",
"amount":100.00 }.

Expected Result:

200 OK; status=SETTLED; availableToSettle=0.00; order status updated to SETTLED.

Actual Result:

Pass/Fail:

Test Script BE 3: Additional Partial Settle When Prior Exists

Test Case ID: SET_TC_003

Description: Add a partial settlement where prior settlements exist.

Preconditions: ORD1002 authorized \$100.00; prior settlements \$40.00 (remaining \$60.00).

Test Steps:

1. POST /api/settlements { "orderId":"ORD1002", "amount":30.00 }.

Expected Result:

200 OK; status=PARTIALLY_SETTLED; availableToSettle=30.00.

Actual Result:

Pass/Fail:

Test Script BE 4: Exceed Remaining Authorized

Test Case ID: SET_TC_004

Description: Attempt to settle more than remaining authorized amount.

Preconditions: After A3, ORD1002 remaining = \$30.00.

Test Steps:

1. POST /api/settlements { "orderId":"ORD1002", "amount":40.00 }.

Expected Result:

422 Unprocessable Entity; { "code": "AMOUNT_EXCEEDS_AVAILABLE",
"availableToSettle":30.00 }.

Actual Result:

Pass/Fail:

Test Script BE 5: No Approved Authorization on File

Test Case ID: SET_TC_005

Description: Settlement should fail if the order has no approved authorization.

Preconditions: ORD2001 exists; no approved authorization.

Test Steps:

1. POST /api/settlements { "orderId":"ORD2001", "amount":25.00 }.

Expected Result:

409 Conflict; { "code": "NO_APPROVED_AUTH" }.

Actual Result:

Pass/Fail:

Test Script BE 6: Order Not Found

Test Case ID: SET_TC_006

Description: Handle settlement for unknown order id.

Preconditions: ORD9999 does not exist.

Test Steps:

1. POST /api/settlements { "orderId":"ORD9999", "amount":10.00 }.

Expected Result:

404 Not Found; { "code":"ORDER_NOT_FOUND" }.

Actual Result:**Pass/Fail:****Test Script BE 7: Amount Validation – zero/negative**

Test Case ID: SET_TC_007

Description: Reject zero or negative amounts.

Preconditions: Any authorized order.

Test Steps:

1. POST /api/settlements { "orderId":"ORD1001", "amount":0 }.
2. POST /api/settlements { "orderId":"ORD1001", "amount":-5 }.

Expected Result:

422 Unprocessable Entity; { "code":"INVALID_AMOUNT" }.

Actual Result:**Pass/Fail:****Test Script BE 8: Amount Precision – > 2 decimals**

Test Case ID: SET_TC_008

Description: Prevent amounts with more than two decimals.

Preconditions: Any authorized order.

Test Steps:

1. POST /api/settlements { "orderId":"ORD1001", "amount":10.999 }.

Expected Result:

422 Unprocessable Entity; { "code":"INVALID_AMOUNT_PRECISION" }.

Actual Result:

422 response returned with INVALID_AMOUNT_PRECISION.

Pass/Fail: Pass

Test Script BE 9: Server/Network Error Handling

Test Case ID: SET_TC_009

Description: API returns 500 error.

Preconditions: Simulate temporary server failure.

Test Steps:

1. POST /api/settlements { "orderId":"ORD1001", "amount":5.00 }.

Expected Result:

500 Internal Server Error; { "code":"SERVER_ERROR" }.

Actual Result:

Pass/Fail:

Test Script BE 10: GET /api/orders (read-only list)

Test Case ID: ORD_TC_001

Description: Fetch list of orders.

Preconditions: API running with ORD1001 and ORD1002.

Test Steps:

1. GET /api/orders.

Expected Result:

200 OK; two rows returned with correct fields.

Actual Result:

Pass/Fail:

Test Script BE 11: GET /api/orders/:id (details)

Test Case ID: ORD_TC_002

Description: Retrieve order details with authorization and settlements.

Preconditions: ORD1002 authorized \$100.00; settlements \$40.00 and \$30.00.

Test Steps:

1. GET /api/orders/ORD1002.

Expected Result:

200 OK with correct order, authorization, settlements, and availableToSettle=30.00.

Actual Result:

Pass/Fail:

Test Script BE 12: GET /health

Test Case ID: SYS_TC_001

Description: Health endpoint sanity check.

Preconditions: API running.

Test Steps:

1. GET /health.

Expected Result:

200 OK with { ok:true, service:"payments-capstone-api",
providerBase:"<beeceptor url>" }.

Actual Result:

Pass/Fail:

Database Connectivity & Functions – Felix

Test Script DB1: Verify Database Connection

Test Case ID: DB_TC_001

Description: Confirm that the SQLite database connects successfully on server start.

Preconditions: Backend server is started.

Test Steps:

1. Run `node server.js`.
2. Observe console output.

Expected Result: Console displays Connected to SQLite database.

Test Script DB2: Table Creation

Test Case ID: DB_TC_002

Description: Verify that the orders table is created on startup.

Preconditions: Database file deleted.

Test Steps:

1. Delete `database.db`.
2. Start server again.
3. Check that `database.db` is recreated.

Expected Result: orders table exists in the new database file.

Test Script DB3: Insert Record via /authorize

Test Case ID: DB_TC_003

Description: Confirm that POST `/authorize` inserts a record into the database.

Preconditions: Database is connected.

Test Steps:

1. Send POST request to `/authorize` with valid data.
2. Open `/orders` or query database directly.

Expected Result: The order appears in the table with correct status (AUTHORIZED or ERROR).

Test Script DB4: Fetch Orders

Test Case ID: DB_TC_004

Description: Verify that GET /orders returns all rows.

Preconditions: At least one record exists in the database.

Test Steps:

1. Send GET request to /orders.
2. Observe response data.

Expected Result: JSON array of orders with correct fields and timestamps.

Test Script DB5: Health Endpoint

Test Case ID: DB_TC_005

Description: Verify that GET /health responds correctly.

Preconditions: Backend is running.

Test Steps:

1. Send GET request to /health.

Expected Result:

```
{
  "ok": true,
  "service": "payments-capstone-api",
  "providerBase":
"https://capstoneproject.free.beeceptor.com/authorize"
}
```

AI Use Disclaimer:

Artificial intelligence tools were utilized in this milestone to assist with formatting and structuring the test scripts and results to align with the provided project template. All content and outcomes were reviewed and finalized by the project team.